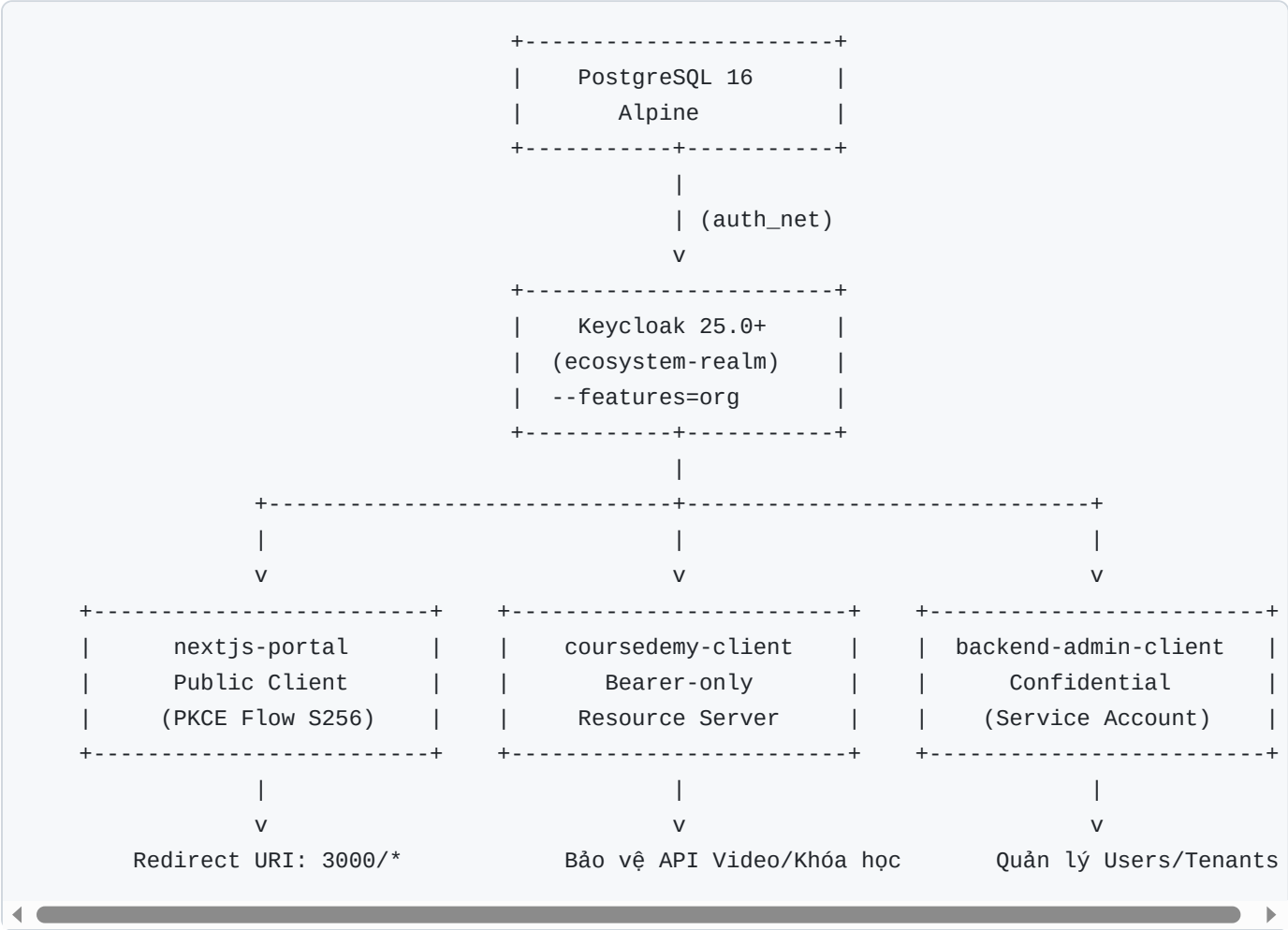


Hệ thống Định danh Tập trung (Central IAM & SSO) cho SaaS Multi-Tenant

Dự án thiết lập toàn bộ hạ tầng Identity and Access Management (IAM) và Single Sign-On (SSO) cho hệ sinh thái SaaS Multi-tenant sử dụng **Keycloak 25.0+** kết hợp cơ sở dữ liệu **PostgreSQL 16 Alpine**.

1. Kiến trúc Hệ sinh thái & Các Thành phần



Chi tiết các Clients trong Realm `ecosystem-realm` :

Client ID	Loại Client	Cơ chế Xác thực (Auth Flow)	Quyền hạn / Mục đích
<code>nextjs-portal</code>	Public	Authorization Code Flow + PKCE (S256)	Frontend SPA Portal cho người dùng đăng nhập SSO. Redirect: <code>http://localhost:3000/*</code> .
<code>coursedemy-client</code>	Bearer-only	JWT Validation (Resource Server)	Backend API bảo vệ tài nguyên học tập của sản phẩm Coursedemy.
<code>vihotask-client</code>	Bearer-only	JWT Validation (Resource Server)	Backend API bảo vệ tài nguyên quản lý công việc của sản phẩm VihoTask.
<code>backend-admin-client</code>	Confidential	Client Credentials (Service Account)	Backend Core Microservice có quyền <code>manage-users</code> , <code>view-users</code> để quản trị người dùng.

2. Cấu trúc Thư mục Hệ Sinh Thái (Ecosystem Monorepo Hub)

```
.
├── apps/                # Các ứng dụng & Client Microservices trong hệ sinh thái
│   ├── coursedemy/      # [Git Submodule] -> github.com/vanhdz74/CourseDemy_v2
│   └── vihotask/        # [Git Submodule] -> github.com/vanhdz74/VihoTask
├── docker-compose.yml   # Khởi chạy PostgreSQL 16 và Keycloak 25+
├── realm-config.json     # File định nghĩa Realm, Clients, Roles, Protocol Mappers & Users
├── .env.example         # File mẫu biến môi trường
├── .env                 # File cấu hình môi trường thực thi
├── test-token.sh        # Script bash kiểm thử cấp token và giải mã JWT claims
├── .gitmodules          # Cấu hình liên kết Git Submodules
└── README.md           # Tài liệu hướng dẫn chi tiết
```

Cách Clone toàn bộ hệ sinh thái kèm các Submodule:

```
git clone --recurse-submodules https://github.com/vanhdz74/Vahiztech.git
cd Vahiztech
```

3. Protocol Mappers & Token Payload Structure

Access Token được cấu hình tự động nhúng các Claims phục vụ Multi-tenancy và cấp phép bản quyền:

- 1. `tenant_id` : Kiểu `String` , định danh Tenant / Tổ chức mà người dùng trực thuộc (ví dụ: `tenant-alpha` , `tenant-beta`).

2. **licenses**: Kiểu Multivalued **String** (JSON Array), thể hiện danh sách sản phẩm/dịch vụ người dùng được phép truy cập (ví dụ: `["coursedemy", "vihotask"]`).
3. **roles**: Mảng các vai trò của người dùng (`org_admin`, `org_member`, `individual_user`).

Mẫu Access Token Payload giải mã:

```
{
  "exp": 1756500000,
  "iat": 1756496400,
  "jti": "8f8ab67c-d6b3-4f96-857c-17937397b91d",
  "iss": "http://localhost:8080/realms/ecosystem-realm",
  "aud": "account",
  "sub": "3df739ef-c5ee-4eb3-b68e-9080b0bb1123",
  "typ": "Bearer",
  "azp": "nextjs-portal",
  "session_state": "ef7d66da-c70e-436f-870a-04b39a3f81e3",
  "scope": "openid email profile tenant-info",
  "sid": "ef7d66da-c70e-436f-870a-04b39a3f81e3",
  "email_verified": true,
  "roles": [
    "org_admin",
    "default-roles-ecosystem-realm"
  ],
  "name": "Alex Admin",
  "preferred_username": "tenant_admin",
  "given_name": "Alex",
  "family_name": "Admin",
  "email": "admin@tenant-alpha.com",
  "tenant_id": "tenant-alpha",
  "licenses": [
    "coursedemy",
    "vihotask"
  ]
}
```

4. Hướng dẫn Khởi chạy & Kiểm thử

Bước 1: Khởi động Containers

```
docker compose up -d
```

Kiểm tra trạng thái containers:

```
docker compose ps
```

Bước 2: Truy cập Giao diện Quản trị Keycloak

- URL: http://localhost:8080
- Master Admin Username: admin (xem trong .env)
- Master Admin Password: admin_master_password_2026 (xem trong .env)
- Realm Quản trị: Chọn realm ecosystem-realm trên menu góc trái trên.

Bước 3: Chạy kịch bản kiểm thử tự động

Chúng tôi đã cung cấp sẵn script test-token.sh để xác minh mọi luồng:

```
./test-token.sh
```

5. Tài khoản Kiểm thử Mặc định (Pre-seeded Users)

Username	Password	Realm Role	tenant_id	licenses
tenant_admin	Admin@123456	org_admin	tenant-alpha	["coursedemy", "vihotask"]
tenant_member	Member@123456	org_member	tenant-alpha	["coursedemy"]
individual_user	User@123456	individual_user	tenant-beta	["vihotask"]

6. Hướng dẫn Tích hợp Code (Code Snippets)

A. Next.js App Router (`next-auth` hoặc OIDC Client với PKCE)

```
// pages/api/auth/[...nextauth].ts hoặc app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth";
import KeycloakProvider from "next-auth/providers/keycloak";

export const authOptions = {
  providers: [
    KeycloakProvider({
      clientId: "nextjs-portal",
      clientSecret: "", // Public Client không cần client secret
      issuer: "http://localhost:8080/realms/ecosystem-realm",
    }),
  ],
  callbacks: {
    async jwt({ token, account, profile }: any) {
      if (account) {
        token.tenant_id = profile?.tenant_id;
        token.licenses = profile?.licenses;
        token.roles = profile?.roles;
      }
      return token;
    },
    async session({ session, token }: any) {
      session.user.tenant_id = token.tenant_id;
      session.user.licenses = token.licenses;
      session.user.roles = token.roles;
      return session;
    },
  },
};
```

B. Microservice (Backend Express / NestJS / Spring Boot) Xác thực JWT

Xác thực Token từ Header `Authorization: Bearer <token>`:

- Xác thực chữ ký bằng JWKS tại URL: `http://localhost:8080/realms/ecosystem-realm/protocol/openid-connect/certs`
- Kiểm tra `req.user.tenant_id` để phân lập dữ liệu Multi-tenancy (Tenant Isolation).
- Kiểm tra `req.user.licenses.includes("coursedemy")` để phân quyền truy cập chức năng.